MODUL 11 – PIG & KAFKA

1.1. Deskripsi Singkat

Apache Pig adalah sebuah platform pemrosesan data paralel yang dirancang untuk menyederhanakan analisis dan pemrosesan data di atas platform Apache Hadoop. Pig menyediakan bahasa pemrograman tingkat tinggi yang disebut Pig Latin, yang memungkinkan pengguna untuk mengekspresikan alur pemrosesan data dalam bentuk script yang mudah dibaca. Dengan menggunakan Pig, pengguna dapat menyatakan alur pemrosesan data (*data flow*) dengan lebih mudah, meningkatkan produktivitas, dan mempercepat waktu pengembangan analisis data dibandingkan dengan penulisan program MapReduce langsung.

Eksekusi Pig script dapat dilakukan dengan dua acara yaitu mode local dan MapReduce. Pada mode local operasi dieksekusi secara langsung pada mesin lokal tempat Pig dijalankan serta tidak melibatkan komponen MapReduce dari ekosistem Hadoop. Pada mode MapReduce, Pig akan memanfaatkan HDFS dan script dijalankan di atas cluster Hadoop.

Apache Kafka adalah sebuah platform streaming data open-source yang digunakan untuk membangun sistem pengiriman dan pengolahan data secara real-time. Berbeda dengan Pig, Kafka bukan merupakan komponen dari Apache Hadoop. Dengan mengintegrasikan Kafka dengan Hadoop, kita dapat membuat *end-to-end data pipeline* yang menangkap, memproses, dan menganalisis aliran data real-time.

Kafka menggunakan model *publish-subscribe* untuk mengirim dan menerima data secara real-time. Producer (*publisher*) mengirimkan pesan ke topik (topic) yang dapat diakses oleh satu atau lebih consumer (*subscriber*).

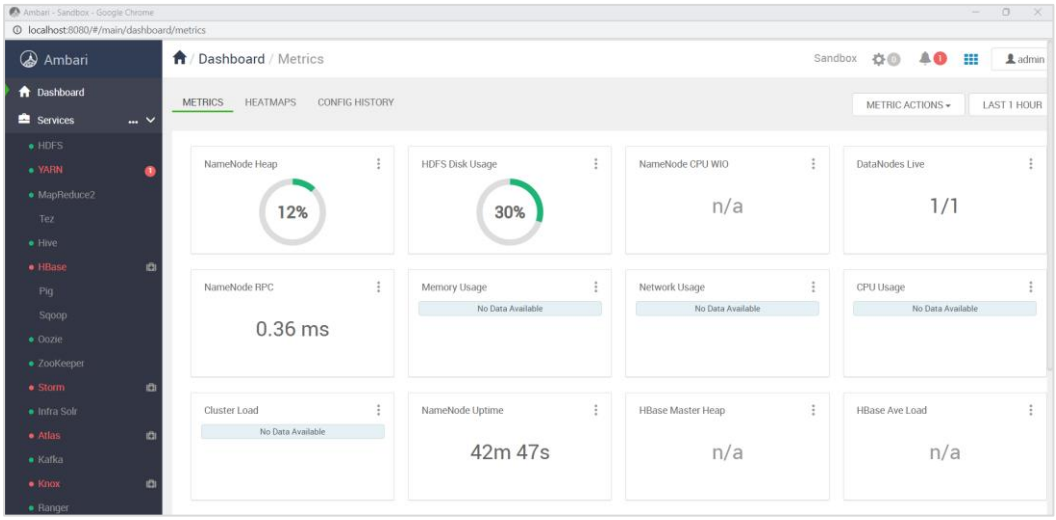
1.2. Tujuan Praktikum

Setelah praktikum pada modul 11 ini, diharapkan mahasiswa mempunyai kompetensi dalam melakukan pengelolaan dan analisis data menggunakan Pig serta melakukan pemrosesan data streaming dengan Kafka.

1.3. Material Praktikum

Persyaratan yang dibutuhkan untuk melakukan praktikum 11 yaitu:

1. HDP yang telah terinstal pada VirtualBox.
2. Ambari dapat diakses



3. Service HDFS, YARN, MapReduce, Zookeeper, Kafka dalam keadaan hidup (*on*) dan dapat digunakan. Untuk menghemat resource, kita bisa matikan service yang tidak kita gunakan pada praktikum modul ini seperti Infra Solr, Oozie, Zeppelin, HBase, Ranger, Spark, Storm, dan lain-lain.
4. MobaXterm untuk mengakses data, mengeksekusi script Pig dan perintah Kafka.

1.4. Kegiatan Praktikum

A. Apache Pig

Pada bagian ini, kita akan mencoba untuk eksekusi Pig script dimulai dari *load* data hingga *store* atau *dump* outputnya dari hasil pemrosesan data. Untuk menghemat resource dan waktu pekerjaan, kita akan melakukan eksekusi Pig dengan mode local. Namun, saat menjalankan script di mode ini, perlu diingat bahwa hasilnya mungkin tidak mencerminkan dengan tepat kinerja dan perilaku yang akan terjadi saat menjalankannya di mode MapReduce. Mode MapReduce menggunakan pemrosesan yang terdistribusi pada cluster Hadoop, yang memungkinkan skalabilitas dan *fault tolerance*.

1. Upload data contoh movies_data.csv pada local system (linux) menggunakan MobaXterm.



2. Jalankan script Pig dengan mode local dengan mekanisme **Grunt atau Interactive mode**.

```
pig -x local
```

```
[root@sandbox-hdp ~]# pig -x local
WARNING: HADOOP_PREFIX has been replaced by HADOOP_HOME. Using value of HADOOP_PREFIX.
23/05/14 19:05:21 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
23/05/14 19:05:21 INFO pig.ExecTypeProvider: Picked LOCAL as the ExecType
2023-05-14 19:05:21,093 [main] INFO  org.apache.pig.Main - Apache Pig version 0.16.0.3.0.1.0-187 (rUnversioned directory
) compiled Sep 19 2018, 10:13:33
2023-05-14 19:05:21,093 [main] INFO  org.apache.pig.Main - Logging error messages to: /root/pig_1684091121091.log
2023-05-14 19:05:21,114 [main] INFO  org.apache.pig.impl.util.Utils - Default bootup file /root/.pigbootup not found
2023-05-14 19:05:21,224 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hado
op file system at: file:///
2023-05-14 19:05:21,419 [main] INFO  org.apache.pig.PigServer - Pig Script ID for the session: PIG-default-0aec803f-0b8d
-43e7-a225-bfdf8e0090e0
2023-05-14 19:05:21,419 [main] WARN  org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set
to false
grunt>
```

3. Jalankan script Pig untuk load data dari movies_data.csv menggunakan PigStorage dengan mendefinisikan urutan field dari data. **Pastikan lokasi file movies_data.csv ada di working directory (sesuaikan path jika lokasi berbeda).**

```
grunt> movies = LOAD 'movies_data.csv' USING PigStorage(',')
as (id,name,year,rating,duration);
```

```
grunt> movies = LOAD 'movies_data.csv' USING PigStorage(',') as (id,name,year,rating,duration);
grunt>
```

4. Jalankan script untuk menampilkan nilai dari **movies**.

```
grunt> dump movies;
```

Perhatikan log ketika script di atas dijalankan, misalkan logical plan (disimpan job DAG), logical plan optimizer, kemudian di-*compile* menjadi job MapReduce (local), dan seterusnya.

```
(1,The Shawshank Redemption,1986,9.2,7560)
(2,Life Is Beautiful,1985,8.6,6300)
(3,Star Wars,1988,8.1,7802)
(4,Batman Begins,1993,8.7,6022)
(5,Jurassic Park,1991,8.4,5420)
(6,Finding Nemo,2004,8.9,4904)
(7,The Terminator,1987,8.4,5623)
(8,Harry Potter and the Deathly Hallows,1987,8.4,7563)
(9,Network,1990,7.2,6244)
(10,My Neighbor Totoro,2004,9.1,6956)
```

5. **Filter data**, misalnya kita ingin memfilter data untuk rating di atas 8.5, maka jalankan perintah sebagai berikut.

```
grunt> movies_greater_than_85 = FILTER movies BY (float)rating
> 8.5;
```

```
grunt> movies_greater_than_85 = FILTER movies BY (float)rating > 8.5;
2023-05-14 19:28:08,516 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan -
LE 1 time(s).
```

```
grunt> dump movies_greater_than_85;
```

```
(1,The Shawshank Redemption,1986,9.2,7560)
(2,Life Is Beautiful,1985,8.6,6300)
(4,Batman Begins,1993,8.7,6022)
(6,Finding Nemo,2004,8.9,4904)
(10,My Neighbor Totoro,2004,9.1,6956)
```

6. Perlu diperhatikan bahwa saat mengeksekusi script Pig, variabel disimpan dalam memori saat pemrosesan (jika koneksi MobaXterm terputus, maka harus mulai dari load data). Pig merupakan *data flow language*, dimana setiap pernyataan operasi data diteruskan dari pernyataan sebelumnya. Data perantara disimpan dalam memori hingga dikonsumsi oleh pernyataan berikutnya atau disimpan menggunakan perintah STORE.

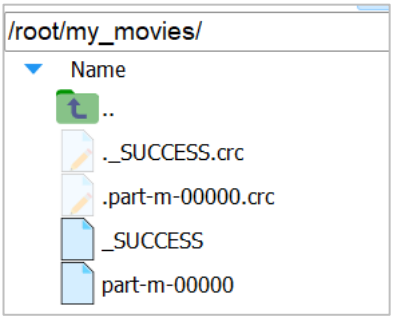
   Misalkan kita akan menyimpan variable **movies_greater_than_85** dalam suatu file, jalankan perintah:

```
grunt> store movies_greater_than_85 into 'my_movies';
```

```
grunt> store movies_greater_than_85 into 'my_movies';
```

   Maka akan terbentuk directory baru pada working directory bernama **my_movies**.

```
grunt> ls
file:/root/.tcshrc<r 1> 129
file:/root/.bashrc<r 1> 251
file:/root/.bash_profile<r 1>   176
file:/root/.bash_logout<r 1>    18
file:/root/anaconda-ks.cfg<r 1> 3302
file:/root/.cshrc<r 1>  100
file:/root/.pig_history<r 1>    497
file:/root/.bash_history<r 1>   13588
file:/root/salaryinfo.txt<r 1>  29098
file:/root/annual-enterprise-survey-2021-fina
file:/root/tpdmr2       <dir>
file:/root/my_movies    <dir>
file:/root/emp.java<r 1>        17611
file:/root/emp_ex.java<r 1>     17677
file:/root/booth.txt<r 1>       372
file:/root/emp2.java<r 1>       17633
file:/root/pig_1684092439510.log<r 1>   2192
file:/root/tpdmr        <dir>
file:/root/movies_data.csv<r 1> 342
file:/root/pg20417.txt<r 1>     185265
```




```
grunt> cat my_movies/part-m-00000;
```

```
grunt> cat my_movies/part-m-00000;
1       The Shawshank Redemption        1986    9.2     7560
2       Life Is Beautiful       1985    8.6     6300
4       Batman Begins   1993    8.7     6022
6       Finding Nemo    2004    8.9     4904
10      My Neighbor Totoro      2004    9.1     6956
```

7. Pada langkah load data sebelumnya, kita hanya mendefinisikan nama field tanpa tipe datanya, sehingga jika kita akan melakukan pemrosesan data, kita harus mendeklarasikan konversi tipe data secara eksplisit, misalnya: `(float)rating`.

   Kita dapat juga mendefinisikan tipe data masing-masing field pada saat load data (langkah 3) dengan perintah:

```
grunt> movies = LOAD 'movies_data.csv' USING PigStorage(',')
as (id:int,name:chararray,year:int,rating:double,
duration:int);
```

```
grunt> movies = LOAD 'movies_data.csv' USING PigStorage(',') as (id:int,name:chararray,year:int,rating:double, duration:
int);
2023-05-14 19:48:39,665 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan - Encountered Warning IMPLICIT_CAST_TO_DOUB
LE 1 time(s).
grunt>
```

8. Kita lakukan filter data dari variable movies yang telah di-*overwrite* dengan nama field dan tipe datanya.

```
grunt> movies_between_9095 = FILTER movies by year > 1990 and
year < 1995;
grunt> movies_starting_with_The = FILTER movies by name
matches 'The.*';
grunt> movies_duration_2hrs = FILTER movies by duration >
7200;
```

```
grunt> movies_between_9095 = FILTER movies by year > 1990 and year < 1995;
2023-05-14 19:53:46,681 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan - Enco
LE 1 time(s).
grunt> movies_starting_with_The = FILTER movies by name matches 'The.*';
2023-05-14 19:53:56,393 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan - Enco
LE 1 time(s).
grunt> movies_duration_2hrs = FILTER movies by duration > 7200;
2023-05-14 19:54:03,596 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan - Enco
LE 1 time(s).
```

Kemudian kita tampilkan masing-masing variable di atas.

```
grunt> dump movies_between_9095;
grunt> dump movies_starting_with_The;
grunt> dump movies_duration_2hrs;
```

```
(4,Batman Begins,1993,8.7,6022)
(5,Jurassic Park,1991,8.4,5420)
```

```
(1,The Shawshank Redemption,1986,9.2,7560)
(7,The Terminator,1987,8.4,5623)
```

```
(1,The Shawshank Redemption,1986,9.2,7560)
(3,Star Wars,1988,8.1,7802)
(8,Harry Potter and the Deathly Hallows,1987,8.4,7563)
```

9. Untuk menampilkan skema dari suatu variable kita dapat menggunakan perintah:

```
grunt> describe movies;
```

```
grunt> DESCRIBE movies;
movies: {id: int,name: chararray,year: int,rating: double,duration: int}
```

10. Perintah FOREACH digunakan untuk menerapkan transformasi data berdasarkan field/kolom. Misalkan kita akan membuat field baru yang dihasilkan dari transformasi data duration dari detik menjadi menit untuk setiap baris data.

```
grunt> movie_duration_mnt = FOREACH movies GENERATE name,
(double)(duration/60);
```

Nilai dari variabel **movie_duration_mnt** akan terdiri dari field name dan field baru bertipe double (durasi movie dalam menit).

```
(The Shawshank Redemption,126.0)
(Life Is Beautiful,105.0)
(Star Wars,130.0)
(Batman Begins,100.0)
(Jurassic Park,90.0)
(Finding Nemo,81.0)
(The Terminator,93.0)
(Harry Potter and the Deathly Hallows,126.0)
(Network,104.0)
(My Neighbor Totoro,115.0)
grunt> describe movie_duration_mnt;
2023-05-14 20:07:28,400 [main] WARN  org.apach
1 time(s).
movie_duration_mnt: {name: bytearray,double}
```

11. Perintah GROUP digunakan untuk mengelompokkan field-field dalam suatu data.

```
grunt> grouped_by_year = group movies by year;
grunt> count_by_year = FOREACH grouped_by_year GENERATE group,
COUNT(movies);
```

```
grunt> grouped_by_year = group movies by year;
2023-05-14 20:12:55,576 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan -
1 time(s).
grunt> count_by_year = FOREACH grouped_by_year GENERATE group, COUNT(movies);
2023-05-14 20:13:05,265 [main] WARN  org.apache.pig.newplan.BaseOperatorPlan -
1 time(s).
```

Kemudian kita tampilkan masing-masing variable di atas.

```
grunt> dump grouped_by_year;
```

```
(1985,{(2,Life Is Beautiful,1985,8.6,6300)})
(1986,{(1,The Shawshank Redemption,1986,9.2,7560)})
(1987,{(8,Harry Potter and the Deathly Hallows,1987,8.4,7563),(7,The Terminator,1987,8.4,5623)})
(1988,{(3,Star Wars,1988,8.1,7802)})
(1990,{(9,Network,1990,7.2,6244)})
(1991,{(5,Jurassic Park,1991,8.4,5420)})
(1993,{(4,Batman Begins,1993,8.7,6022)})
(2004,{(10,My Neighbor Totoro,2004,9.1,6956),(6,Finding Nemo,2004,8.9,4904)})
```

```
grunt> dump count_by_year;
```

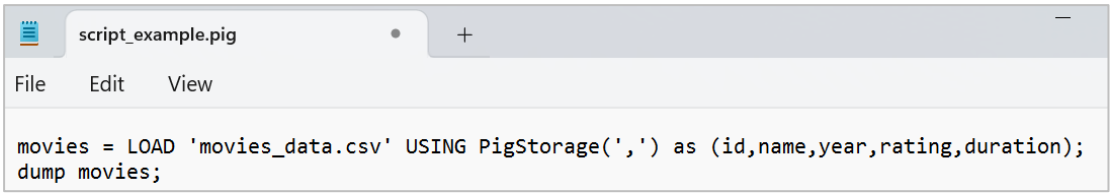
```
(1985,1)
(1986,1)
(1987,2)
(1988,1)
(1990,1)
(1991,1)
(1993,1)
(2004,2)
```

12. Untuk melakukan pengurutan data kita dapat menggunakan perintah ORDER. Jika kita ingin menampilkan beberapa baris data teratas, kita dapat menggunakan perintah LIMIT.

```
grunt> desc_movies_by_year = ORDER movies BY year DESC;
grunt> asc_movies_by_year = ORDER movies by year ASC;
grunt> top5_latest_movies = LIMIT desc_movies_by_year 5;
```

```
(6,Finding Nemo,2004,8.9,4904)
(10,My Neighbor Totoro,2004,9.1,6956)
(4,Batman Begins,1993,8.7,6022)
(5,Jurassic Park,1991,8.4,5420)
(9,Network,1990,7.2,6244)
```

13. Langkah-langkah di atas merupakan eksekusi Pig script dengan mekanisme Interactive. Untuk menjalankan eksekusi dalam **Batch mode**, kita dapat menuliskan script pada suatu file Pig (.pig).



Kemudian jalankan perintah di bawah ini pada local system (bukan grunt):

```
pig -x local script_example.pig
```

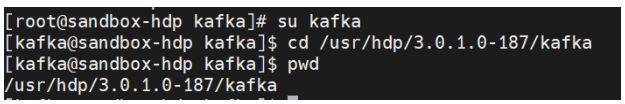
Seharusnya output dari perintah tersebut akan menampilkan data movies.

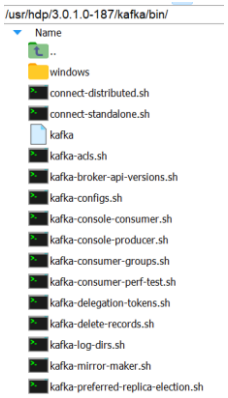
B. Apache Kafka

Pada bagian ini kita akan menjalankan model *publisher-subscribe* untuk data streaming menggunakan Apache Kafka. Pastikan service Kafka dan Zookeeper pada Hadoop dalam keadaan hidup (*on*). Bash file (**.sh**) Apacha Kafka pada sistem HDP ada pada path: **/usr/hdp/3.0.1.0-187/kafka** (lokasi folder bisa berbeda tergantung versi HDP).

1. Masuk ke directory **/usr/hdp/3.0.1.0-187/kafka** dan pastikan bash file Kafka berada pada sub directory **bin** di dalam directory tersebut.

```
su kafka
cd /usr/hdp/3.0.1.0-187/kafka
```





2. Konfigurasi Kafka Server dapat dilihat pada file **server.properties** di directory **/etc/kafka/3.0.1.0-187/0/** (untuk melihat konten file tersebut, bisa diunduh dan dibuka di mesin host).

```
port=6667
producer.metrics.enable=false
producer.purgatory.purge.interval.requests=10000
queued.max.requests=500
replica.fetch.max.bytes=1048576
replica.fetch.min.bytes=1
replica.fetch.wait.max.ms=500
replica.high.watermark.checkpoint.interval.ms=5000
replica.lag.max.messages=4000
replica.lag.time.max.ms=10000
replica.socket.receive.buffer.bytes=65536
replica.socket.timeout.ms=30000
sasl.enabled.mechanisms=GSSAPI
sasl.mechanism.inter.broker.protocol=GSSAPI
security.inter.broker.protocol=PLAINTEXT
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
socket.send.buffer.bytes=102400
ssl.client.auth=none
ssl.key.password=
ssl.keystore.location=
ssl.keystore.password=
ssl.truststore.location=
ssl.truststore.password=
zookeeper.connect=sandbox-hdp.hortonworks.com:2181
zookeeper.connection.timeout.ms=25000
zookeeper.session.timeout.ms=30000
zookeeper.sync.time.ms=2000
```

3. Buat Topic dengan nama **tpd_logs** dengan perintah:

```
bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1 --topic tpd_logs
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-topics.sh --create --zookeeper localhost:2181 --replication-factor 1 --partitions 1
 --topic tpd_logs
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issu
es it is best to use either, but not both.
Created topic "tpd_logs".
```

4. Tampilkan daftar Topics yang telah dibuat dengan perintah:

```
bin/kafka-topics.sh --list --zookeeper localhost:2181
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-topics.sh --list --zookeeper localhost:2181
ATLAS_ENTITIES
ATLAS_HOOK
__consumer_offsets
tpd_logs
```

5. Tampilkan informasi konfigurasi suatu Topic misal jumlah partition, replica, partition ID, current offset, dan lain-lain dengan perintah:

```
bin/kafka-topics.sh --describe --zookeeper localhost:2181 --topic tpd_logs
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-topics.sh --describe --zookeeper localhost:2181 --topic tpd_logs
Topic:tpd_logs  PartitionCount:1        ReplicationFactor:1     Configs:
        Topic: tpd_logs Partition: 0    Leader: 1001    Replicas: 1001  Isr: 1001
```

6. Suatu Producer mem-*publish* messages ke Topic **tpd_logs** dengan perintah sebagai berikut:

```
bin/kafka-console-producer.sh --broker-list sandbox-hdp.hortonworks.com:6667 --topic tpd_logs
> ini adalah log 1
> log 2 masuk
> log 3 created
>
```

Listener pada konfigurasi Kafka Server ada di alamat `sandbox-hdp.hortonworks.com:6667` (cek langkah 2).

```
listeners=PLAINTEXT://sandbox-hdp.hortonworks.com:6667
```

7. Buka **tab baru MobaXterm** yang akan berperan sebagai Consumer yang men-*subscribe* messages Topic **tpd_logs** dengan masuk ke directory Kafka pada langkah 1 dan kemudian menjalankan perintah sebagai berikut:

```
bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic tpd_logs --from-beginning
```

Setiap message yang dikirim oleh Producer akan secara ***"real-time"*** dikonsumsi oleh Consumer.

```
[kafka@sandbox-hdp kafka]$ bin/kafka-console-producer.sh --broker-list sandbox-hdp.hortonworks.com:6667 --topic tpd_logs
>ini adalah log 1
>log 2 masuk
>
```

```
[kafka@sandbox-hdp kafka]$ bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic tpd_logs --from-beginning
Using the ConsoleConsumer with old consumer is deprecated and will be removed in a future major release. Consider using
the new consumer by passing [bootstrap-server] instead of [zookeeper].
ini adalah log 1
log 2 masuk
```

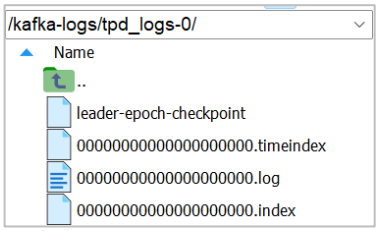
Gunakan Ctrl+C untuk keluar dari shell Producer atau Consumer.

Jika Consumer hanya ingin mengkonsumsi message baru saja, tidak dari awal, maka gunakan perintah:

```
bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic tpd_logs
```

```
^C[kafka@sandbox-hdp kafka]$ bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic tpd_logs
Using the ConsoleConsumer with old consumer is deprecated and will be removed in a future major release. Consider using
the new consumer by passing [bootstrap-server] instead of [zookeeper].
log 9
this log 10
insert into log
```

8. Jalankan langkah 6 dan 7 terus menerus secara simultan untuk melihat komunikasi di mana Producer (publisher) mengirim message ke Topics, dan Consumer (subscriber) menerima message dari topik-topik yang mereka minati. Data message tersebut tersimpan di directory: **/kafka-logs/tpd_logs-0/**



Setelah produser berhasil menulis suatu message, file log segmen `00000000000000000000.log` akan bertambah besar.

9. Kita dapat menggunakan **Kafka Connect** untuk import/export data Kafka. Misalkan, Producer merupakan suatu aplikasi yang menyimpan log data dalam sebuah file dan akan dikirim ke Kafka Server untuk dikonsumsi oleh Consumer.
    - Siapkan file log **log_data.txt** di **/home/kafka/**

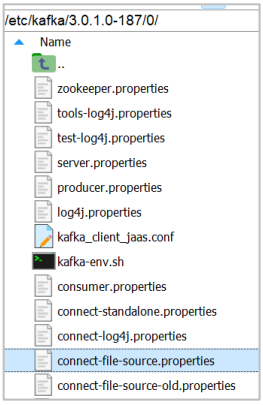
```
[2023-05-22 10:15:32] INFO: Starting Kafka broker with ID 1234
[2023-05-22 10:15:34] INFO: Registered broker 1234 successfully with ZooKeeper
[2023-05-22 10:15:35] INFO: Kafka broker 1234 initialized and ready to accept connections
[2023-05-22 10:15:40] WARN: High CPU usage detected. Consider optimizing consumer groups.
[2023-05-22 10:15:45] ERROR: Failed to process message from topic "my_topic". Offset: 5678
[2023-05-22 10:15:46] INFO: Replication factor for topic "my_topic" increased to 3
[2023-05-22 10:15:50] INFO: Kafka broker 1234 shutting down gracefully
[2023-05-22 10:15:52] INFO: Kafka broker 1234 successfully stopped
```

- File konfigurasi **connect-file-source.properties** di **/etc/kafka/3.0.1.0-187/0/** (rename file sebelumnya menjadi connect-file-source-old.properties)

```
name=local-file-source
connector.class=FileStreamSource
tasks.max=1
file=/home/kafka/log_data.txt
topic=tpd_logs
```

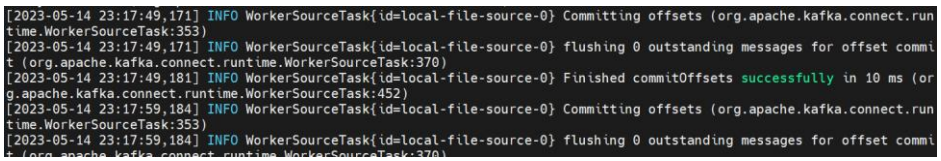
- File konfigurasi **connect-standalone.properties** di path yang sama (rename file sebelumnya menjadi connect- standalone -old.properties), ubah baris:

```
bootstrap.servers=sandbox-hdp.hortonworks.com:6667
```



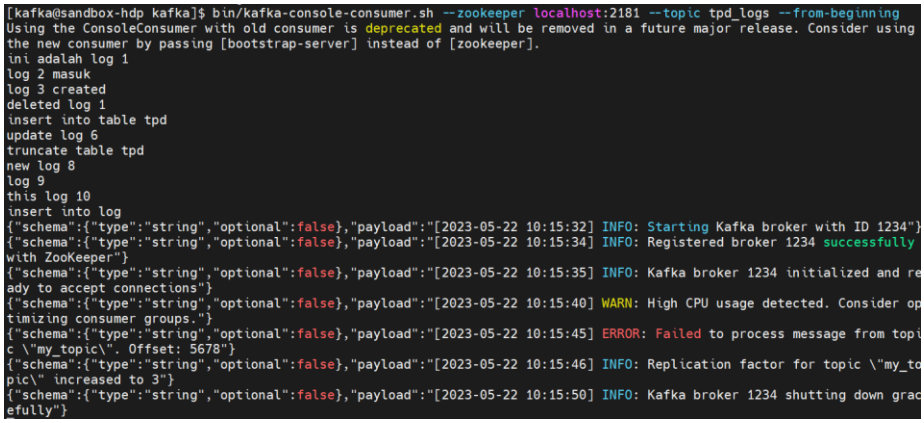
Producer dapat menjalankan perintah:

```
bin/connect-standalone.sh config/connect-
standalone.properties config/connect-file-source.properties
config/connect-file-sink.properties
```



Consumer dapat menjalankan perintah:

```
bin/kafka-console-consumer.sh --zookeeper localhost:2181 --
topic tpd_logs --from-beginning
```



1.5. Penugasan

Kerjakan sesuai dengan yang dijelaskan pada bagian Kegiatan Praktikum.

1. Lakukan pemrosesan data menggunakan Pig script (interactive atau batch) untuk menghitung jumlah tiap kata pada tulisan di bawah ini.

```
Hello students, how are you today?
I hope you are having a great day.
Today is a good day to learn Pig Latin.
Pig Latin is a data flow scripting language used for big data processing.
It is widely used in the Apache Hadoop ecosystem.
```

   - Dengan melengkapi script di bawah ini:

```
lines = LOAD 'text_data.txt' AS (line:chararray);
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) as word;
grouped = GROUP … BY … ;
word_count = … ;
DUMP word_count;
```

   - Screenshot output dari script tersebut.

2. Screenshot output messages dari Producer ke Consumer menggunakan Apache Kafka pada langkah 7 dan 9.